HyperV Agent Frontend Services Architecture

📋 Przegląd

Frontend aplikacji HyperV LocalManagement został zrefaktoryzowany z modułową architekturą serwisów, która zapewnia pełne pokrycie API dokumentowanego w `HYPERV-AGENT-API-FULL-DOCUMENTATION.md`.

🏗 Struktura Serwisów

`baseService.ts`

Podstawowy serwis zawierający wspólne funkcjonalności:

- **`fetchApi()`** - funkcja do wykonywania zapytań HTTP z obsługą błędów
- **`ApiError`** - klasa błędów API z kodem statusu i szczegółami

`vmService.ts`

Kompletne zarządzanie maszynami wirtualnymi:

- **Lifecycle**: `startVm()`, `stopVm()`, `pauseVm()`, `resumeVm()`, `shutdownVm()`, `terminateVm()`, `saveVm()`
- **Snapshots**: `getVmSnapshots()`, `createVmSnapshot()`, `deleteVmSnapshot()`, `revertToSnapshot()`
- **Storage**: `getVmStorageDevices()`, `addStorageDevice()`, `removeStorageDevice()`, `getVmStorageControllers()`
- **Configuration**: `configureVm()`, `getVmProperties()`

`containerService.ts`

Zarządzanie kontenerami:

- **Lifecycle**: `startContainer()`, `stopContainer()`, `pauseContainer()`, `resumeContainer()`, `terminateContainer()`
- **Management**: `createContainer()`, `getContainers()`, `getContainer()`, `deleteContainer()`

`storageService.ts`

Operacje storage i VHD:

- **VHD Operations**: `createVhd()`, `attachVhd()`, `detachVhd()`, `resizeVhd()`, `compactVhd()`, `convertVhdFormat()`
- **Metadata**: `getVhdMetadata()`, `updateVhdMetadata()`
- **VHD Sets**: `convertToVhdSet()`, `getVhdSetInfo()`, `createVhdSetSnapshot()`
- **Change Tracking**: `enableChangeTracking()`, `getVirtualDiskChanges()`

`networkService.ts`

Zarządzanie sieciami:

- **Networks**: `createNatNetwork()`, `deleteNetwork()`, `getNetworkProperties()`
- **Endpoints**: `createNetworkEndpoint()`, `deleteNetworkEndpoint()`, `getEndpointProperties()`

`jobService.ts`

Monitorowanie zadań storage:

- **Jobs**: `getStorageJobs()`, `getStorageJobDetails()`, `getJobAffectedElements()`
- **Control**: `cancelStorageJob()`, `deleteStorageJob()`

`healthService.ts`

Monitorowanie zdrowia serwisów:

- **Health**: `getHealthStatus()`, `getServiceInfo()`

`hypervService.ts`

Główny agregator eksportujący wszystkie serwisy z zachowaniem kompatybilności wstecznej.

🎯 Użycie w Komponencie

```
// Import konkretnego serwisu
import { vmService, containerService } from '../services/hypervService';

// Lub import wszystkich funkcji (kompatybilność wsteczna)
import * as api from '../services/hypervService';

// Przykład użycia
const handleStartVm = async (vmName: string) => {
    try {
        await vmService.startVm(vmName);
        addNotification('success', 'VM started successfully');
    } catch (error: any) {
        addNotification('error', error.message);
    }
};
```

🔧 Interfejsy UI

`VirtualMachinesPage.tsx`

- ☑ **Start/Stop/Pause VM**: Pełna kontrola lifecycle maszyn wirtualnych
- ☑ **Create VM**: Tworzenie nowych maszyn WMI/HCS
- ☑ **VM Properties**: Wyświetlanie CPU/Memory/Status

`ContainersPage.tsx` ⭐ NOWY

- ☑ **Container Lifecycle**: Start/Stop/Pause/Resume/Terminate
- ☑ **Create Container**: Tworzenie kontenerów z konfiguracją
- ☑ **Delete Container**: Usuwanie kontenerów
- ☑ **HCS/WMI Support**: Obsługa obu środowisk

Aktualizacje nawigacji:

- ☑ **SideNav**: Dodano link do kontenerów z ikoną `ContainerIcon`
- ☑ **App.tsx**: Dodano routing `/containers`

📊 Pokrycie API

Grupa endpointów	Status	Implementacja
Service Management	☑ Kompletne	`healthService.ts`
VM Management	☑ Kompletne	`vmService.ts`
Container Management	☑ Kompletne	`containerService.ts`
Network Management	☑ Kompletne	`networkService.ts`
Storage Management	☑ Kompletne	`storageService.ts`
Job Management	☑ Kompletne	`jobService.ts`

Total: **100% pokrycia API** - wszystkie endpointy z dokumentacji API zostały zaimplementowane w odpowiednich serwisach.

🚀 Korzyści Architektury

1. **Modułowość** - każdy serwis odpowiada za swoją domenę
2. **Type Safety** - pełne typowanie TypeScript dla wszystkich operacji
3. **Error Handling** - ujednolicona obsługa błędów przez `ApiError`
4. **Reusability** - serwisy można łatwo używać w różnych komponentach
5. **Maintainability** - łatwiejsze utrzymanie i rozwijanie funkcjonalności
6. **Testing** - każdy serwis można testować niezależnie

🔄 Migration Guide

Stare użycie:

```
import { getVms, startVm } from '../services/hypervService';
```

Nowe użycie (zalecane):

```
import { vmService } from '../services/hypervService';
// lub
import * as vmService from '../services/vmService';
```

Stary kod nadal działa dzięki zachowaniu kompatybilności wstecznej w `hypervService.ts`.